

INTRODUCTION TO SOFTWARE DESIGN

Giới thiệu

Phân tích đặc tả yêu cầu phần mềm (SRA):

-  Không chỉ là việc thu thập yêu cầu mà là quá trình "chuyển ngữ" từ ngôn ngữ kinh doanh (mong muốn của khách hàng) sang ngôn ngữ kỹ thuật.
-  Trả lời câu hỏi **WHAT**

Thiết kế phần mềm (Software design):

-  Quy trình hình dung và thiết kế các giải pháp phần mềm.
 -  Xây dựng khung xương (architecture) cho hệ thống.
 -  Tối ưu hóa cấu trúc mã nguồn để dễ bảo trì, dễ kiểm thử và có tính linh hoạt cao.
-  Trả lời câu hỏi **HOW**

Thiết kế phần mềm

- ☕ Là một chuỗi các bước có tính toán để chuyển hóa requirements thành sản phẩm thực tế
 - 🌿 Cho phép đội ngũ phát triển giải thích được **tại sao** thành phần này lại tồn tại và nó đóng góp gì vào mục tiêu chung của hệ thống.
- ☕ Cung cấp cái nhìn trực quan về cấu trúc hệ thống trước khi bắt tay vào xây dựng
 - 🌿 Bắt đầu từ cái nhìn **tổng thể** (luồng dữ liệu chính, giao diện người dùng)
 - 🌿 Tinh chỉnh dần từng **chi tiết** (thuật toán, cấu trúc dữ liệu, các hàm xử lý lỗi và kết nối dữ liệu)
 - 🌿 Giúp phát hiện lỗi **logic** và các điểm nghẽn tiềm tàng ngay từ giai đoạn **trên giấy**, tiết kiệm chi phí sửa chữa

Thiết kế phần mềm

02 hướng tiếp cận chính tùy vào đặc thù của hệ thống

- ☕ Phần mềm hướng người dùng (**User-centric**):
 - 🌿 Tập trung vào tính tương tác và thẩm mỹ.
 - 🌿 Sử dụng **Storyboard** (kịch bản người dùng) để hình dung hành trình của khách hàng và **UX Design** để đảm bảo phần mềm dễ sử dụng, thân thiện.
- ☕ Phần mềm tự động hoàn toàn (**Automated systems**):
 - 🌿 Tập trung vào logic xử lý ngầm, tính chính xác và hiệu suất.
 - 🌿 Chủ yếu sử dụng Biểu đồ luồng (**Flowchart**) hoặc Mô tả trình tự để thể hiện cách dữ liệu được xử lý giữa các thành phần hệ thống mà không cần sự can thiệp của con người.

Thiết kế phần mềm

- ☕ **Kết quả:** bộ hồ sơ thiết kế (Design Document).
- ☕ Phân loại:
 - 🌿 Độc lập nền tảng (**Platform-independent**): Thiết kế logic, cấu trúc dữ liệu tổng quát (có thể áp dụng cho mọi ngôn ngữ lập trình hay hệ điều hành).
 - 🌿 Đặc thù nền tảng (**Platform-specific**): Chỉ rõ công nghệ sử dụng (ví dụ: thiết kế riêng cho React Native, AWS, hay SQL Server), giúp lập trình viên có thể bắt tay vào viết code ngay lập tức.

Thiết kế phần mềm

- ☕ Tập trung vào khả năng (**Capabilities**): không chỉ giải quyết bài toán của hệ thống mà còn xác định hệ thống **mạnh** đến đâu.
 - 🌿 Không chỉ là *Thanh toán được* mà là *Thanh toán được cho 1 triệu người cùng lúc* hoặc *Thanh toán an toàn theo chuẩn quốc tế*.
 - 🌿 Bảo mật dữ liệu theo tiêu chuẩn quốc tế (**PCI DSS**), chống rò rỉ thông tin ngay cả khi máy chủ bị tấn công.
- ☕ **Sáng tạo và Đa dạng giải pháp**
 - 🌿 Với cùng một yêu cầu *Lưu trữ dữ liệu*, nhóm A có thể chọn SQL để đảm bảo tính nhất quán, nhóm B chọn NoSQL để tối ưu tốc độ. Cả hai đều đúng, nhưng cách tiếp cận khác nhau.

Thiết kế phần mềm

Thiết kế phần mềm không phải là tìm kiếm một giải pháp hoàn hảo về mọi mặt, mà là tìm kiếm một **điểm cân bằng tối ưu** giữa các yếu tố xung đột.

- ☕ Sự đánh đổi (**Trade-offs**):
 - 🌿 Chọn **tốc độ** hay chọn **bảo mật**?
 - 🌿 Chọn **chi phí rẻ** hay chọn **khả năng mở rộng**?
- ☕ Phụ thuộc vào môi trường thực thi
 - 🌿 Mobile: tiết kiệm pin, tối ưu hóa dung lượng bộ nhớ (RAM) và băng thông mạng
 - 🌿 Web: tốc độ phản hồi ban đầu, tính tương thích giữa các trình duyệt và SEO

Phân tích vs Thiết kế

Đặc điểm	Phân tích (Analysis)	Thiết kế (Design)
Mục tiêu	Hiểu rõ vấn đề	Xây dựng giải pháp
Tính chất	Khám phá, logic	Sáng tạo, kỹ thuật
Kết quả	Tài liệu đặc tả (SRS)	Tài liệu thiết kế (SDD)
Sự biến đổi	Thường cố định theo nghiệp vụ	Biến đổi linh hoạt theo công nghệ

CÁC NGUYÊN LÝ THIẾT KẾ PHẦN MỀM

Mục đích: Quản lý sự phức tạp, giảm rủi ro lỗi và công sức.

Kết quả thiết kế

- 1 Thiết kế kiến trúc (Architectural Design):** Mức trừu tượng cao nhất.
 -  **VD:** hệ thống gồm các thành phần lớn như Server, Client, Database và cách chúng kết nối (HTTP, WebSockets, gRPC).
- 2 Thiết kế cấp cao (High-Level Design):** Phân rã thành các hệ thống con và mô-đun.
 -  **VD:** Server có mô-đun "Thanh toán", mô-đun "Quản lý người dùng"
- 3 Thiết kế chi tiết (Detailed Design):** Đi sâu vào cài đặt chi tiết của các mô-đun.
 -  Cung cấp *bản hướng dẫn lắp ráp* chính xác để lập trình viên có thể viết code mà không cần hỏi lại ý tưởng

Software Design Principles

Problem
Partitioning



Modularity



Abstraction



Top Down &
Bottom up
Strategy



Phân chia vấn đề (Problem Partitioning)

Chia để trị: chia một bài toán lớn, phức tạp thành các phần nhỏ, riêng biệt để dễ dàng kiểm soát.

- ☕ **Lợi ích:**
 - 🌿 Hệ thống trở nên *đơn giản hơn* khi cần sử dụng hoặc thay đổi một tính năng cụ thể.
 - 🌿 *Dễ kiểm thử:* Có thể kiểm tra từng phần nhỏ thay vì phải chạy toàn bộ hệ thống khổng lồ.
 - 🌿 *Dễ mở rộng:* Khi muốn thêm tính năng, chỉ cần tác động vào mô-đun liên quan.
- ☕ **Lưu ý:** chia nhỏ quá mức sẽ dẫn đến sự phức tạp trong việc kết nối các thành phần, tạo ra thách thức về tương tác hệ thống.

Trừu tượng hóa (Abstraction)

- ☕ Kỹ thuật **lọc bỏ** các chi tiết **phức tạp** để tập trung vào các **đặc điểm quan trọng nhất** của đối tượng.
- ☕ **Trừu tượng hóa chức năng:**
 - 🌿 Coi một mô-đun như một hộp đen: người dùng chỉ cần biết đầu vào và đầu ra mà không cần quan tâm đến thuật toán bên trong.
 - 🌿 Đây là nền tảng của các phương pháp lập trình *truyền thống*.
- ☕ **Trừu tượng hóa dữ liệu:**
 - 🌿 Che giấu cách thức lưu trữ và tổ chức dữ liệu. Người dùng tương tác với dữ liệu thông qua các hành vi (phương thức).
 - 🌿 Đây là linh hồn của phương pháp thiết kế *hướng đối tượng*.

Tính mô-đun hóa (Modularity)

- ☕ Chia phần mềm thành các đơn vị rời rạc, có tên gọi và định danh riêng.
- ☕ Các tiêu chuẩn của một mô-đun tốt:
 - 🌿 **Tính đơn nhất:** Mỗi mô-đun chỉ thực hiện một nhiệm vụ duy nhất và rõ ràng.
 - 🌿 **Dễ tái sử dụng:** Có thể mang mô-đun từ dự án này sang dự án khác.
 - 🌿 **Nguyên lý Đơn giản:** Việc sử dụng mô-đun phải đơn giản hơn việc tạo ra nó; bên ngoài đơn giản hơn bên trong.
 - 🌿 **Độc lập:** Có khả năng biên dịch và kiểm tra riêng lẻ mà không phụ thuộc quá nhiều vào các phần khác.

Tính mô-đun hóa (Modularity)

Ưu điểm:

- 🍷 Hỗ trợ làm việc nhóm hiệu quả, cho phép nhiều người cùng phát triển đồng thời.
- 🍷 Giúp việc quản lý tiến độ và phát hiện lỗi trở nên trực quan hơn.
- 🍷 Tạo ra cấu trúc mã nguồn có tổ chức, dễ hiểu và dễ bảo trì.

Nhược điểm:

- 🍷 Có thể gây lãng phí tài nguyên như tăng thời gian thực thi hoặc tốn dung lượng lưu trữ.
- 🍷 Tăng độ phức tạp trong khâu liên kết các thành phần và yêu cầu hệ thống tài liệu chi tiết hơn.

Thiết kế Mô-đun (Modular Design)

- ☕ Độc lập chức năng (**Functional Independence**):
 - 🌿 Mục tiêu tối thượng của thiết kế. Một mô-đun nên làm một việc duy nhất và ít giao tiếp thừa với bên ngoài.
 - 🌿 Giúp ngăn chặn tình trạng *lỗi dây chuyền* khi một phần của hệ thống gặp sự cố.
- ☕ Hai thước đo quan trọng:
 - 🌿 **Cohesion** (Độ kết dính): Đo lường sức mạnh bên trong mô-đun (Càng *cao* càng tốt).
 - 🌿 **Coupling** (Độ kết ghép): Đo lường sự phụ thuộc giữa các mô-đun (Càng *thấp* càng tốt).
- ☕ Che giấu thông tin (**Information Hiding**):
 - 🌿 Giới hạn quyền truy cập vào dữ liệu nội bộ, giúp bảo vệ logic cốt lõi và giảm thiểu rủi ro khi thay đổi mã nguồn trong tương lai.

STRATEGY OF DESIGN

Top-Down

- ☕ **Triết lý:** tập trung vào mục tiêu hệ thống trước khi sa đà vào chi tiết.
- ☕ **Thực hiện:** Phân rã hệ thống thành các hệ thống con (**sub-systems**).
- ☕ **Ưu điểm:** Dễ dàng nhận diện các thành phần dư thừa hoặc thiếu sót trong cấu trúc tổng thể.
- ☕ **Thách thức:** Có thể bỏ lỡ các khó khăn kỹ thuật ở mức thấp cho đến khi bắt tay vào làm chi tiết.

STRATEGY OF DESIGN

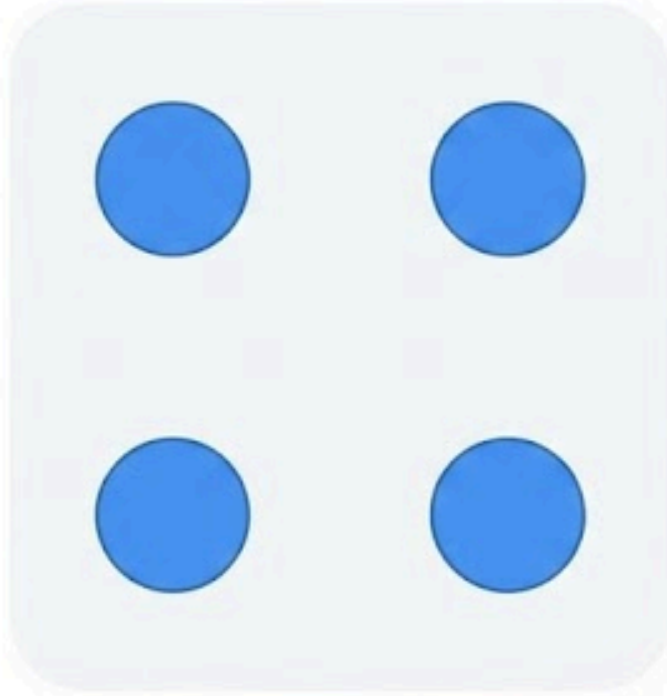
Bottom-Up Approach

- ☕ **Triết lý:** Xây dựng nền móng vững chắc. Tập trung vào các đơn vị xử lý cụ thể.
- ☕ **Thực hiện:** Kết hợp các mô-đun cơ bản (**primitive components**) để tạo ra các cụm chức năng lớn hơn.
- ☕ **Ưu điểm:** Tăng khả năng tái sử dụng (**reusability**) và dễ kiểm thử ở mức độ thấp (**unit testing**).
- ☕ **Thách thức:** Khó khăn trong việc tích hợp các mảnh nhỏ thành một khối thống nhất và khớp với mục tiêu ban đầu.

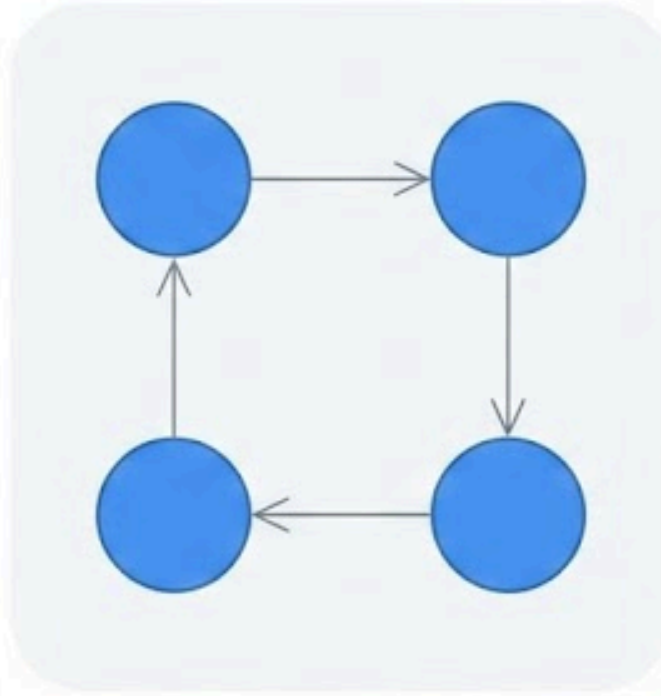
Module Coupling

- ☕ Là thước đo mức độ **phụ thuộc** giữa các mô-đun.
- ☕ **Mục tiêu:** Kết ghép lỏng (*Low/Loose coupling*).
 - 🌿 Một thay đổi trong mô-đun A ít ảnh hưởng đến mô-đun B.
 - 🌿 Tăng tính linh hoạt, dễ thay thế các thành phần.
- ☕ **Tránh:** Kết ghép chặt (*High/Tight coupling*).
 - 🌿 Gây ra *hiệu ứng domino* khi sửa lỗi.
 - 🌿 Hệ thống trở nên cứng nhắc và khó hiểu.

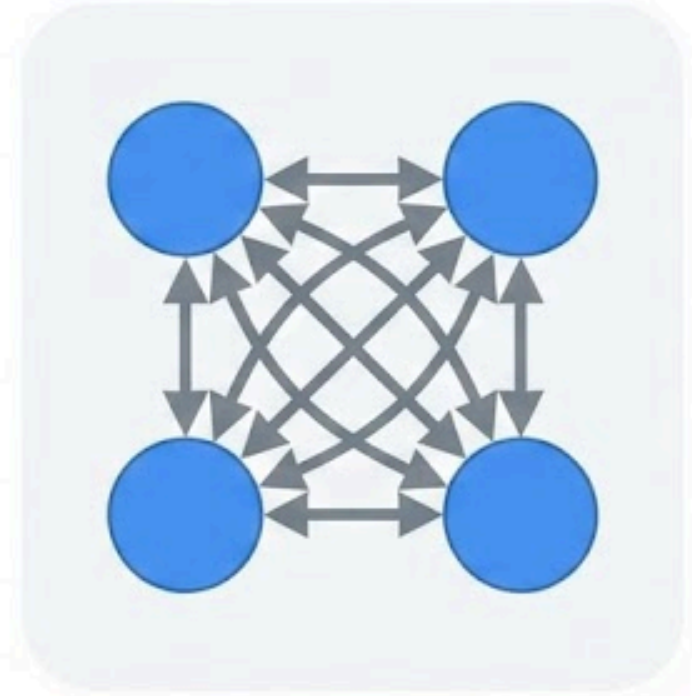
Coupling



(a) Uncoupled: no dependencies



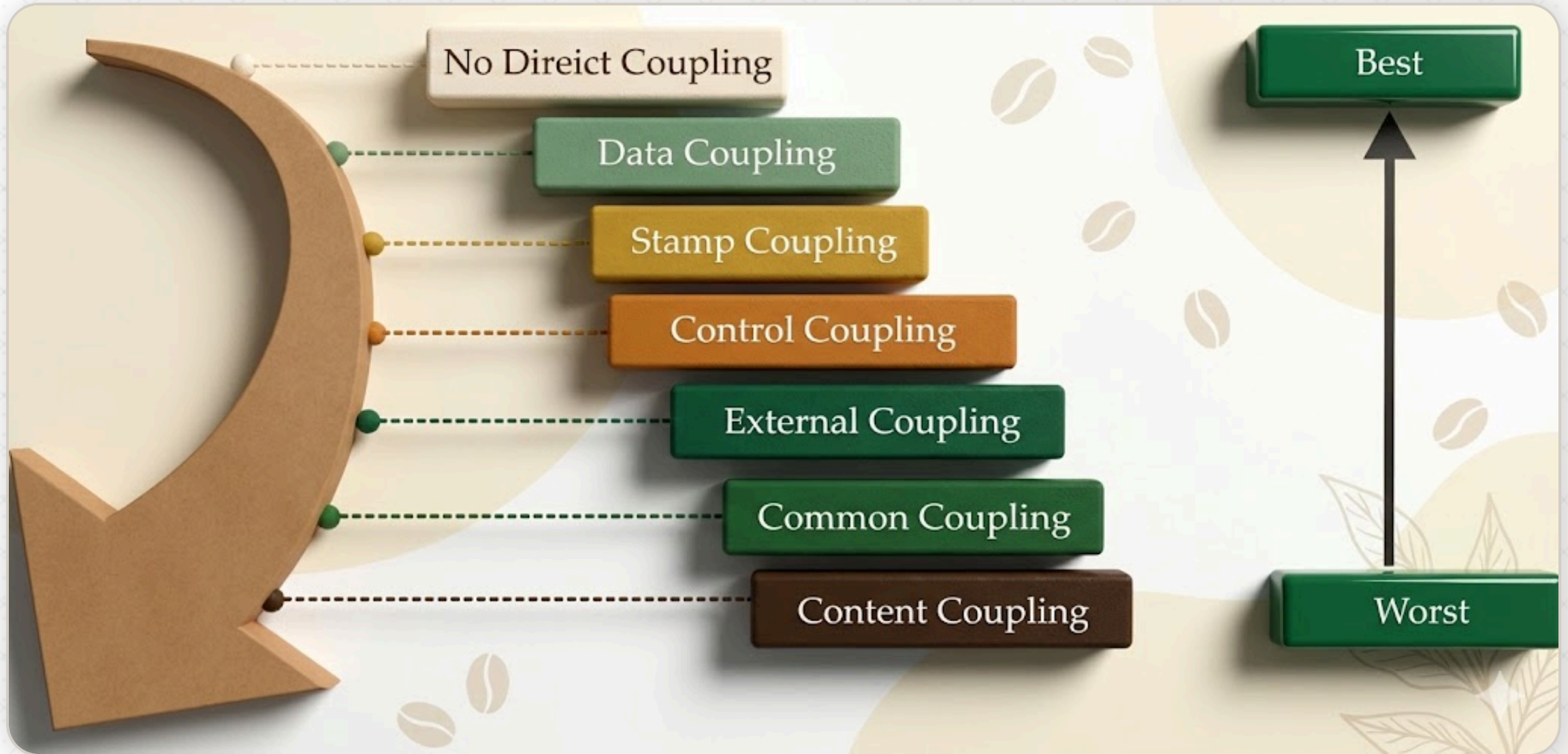
(b) Loosely Coupled:
Some dependencies



(c) Highly Coupled:
Many dependencies

Các loại Kết ghép (Coupling)

- 1 **No Direct Coupling:** Lý tưởng nhất, các mô-đun không biết về nhau.
- 2 **Data Coupling:** Truyền các tham số đơn lẻ (VD: `int`, `string`).
- 3 **Stamp Coupling:** Truyền cả một cấu trúc dữ liệu (Object/Struct) dù mô-đun nhận chỉ cần một phần dữ liệu đó.
- 4 **Control Coupling:** Mô-đun này điều khiển logic của mô-đun kia bằng cách truyền **cờ** (`flags`).
- 5 **External Coupling:** Phụ thuộc vào các yếu tố bên ngoài (File, API, Hardware).
- 6 **Common Coupling:** Sử dụng chung vùng nhớ toàn cục (**Global variable**) - cực kỳ nguy hiểm.
- 7 **Content Coupling:** Một mô-đun *đọc/trộm* hoặc sửa trực tiếp biến nội bộ của mô-đun khác.
 - 🕸 Phá vỡ hoàn toàn nguyên lý *Information Hiding*. Khi mô-đun bị *đọc/trộm* thay đổi dù chỉ một chút, mô-đun *đọc/trộm* sẽ hỏng hoàn toàn.



Content Coupling

```
// Lớp bị xâm phạm
class Inventory {
    // Biến nội bộ nhưng để public hoặc không có getter/setter bảo vệ
    public int stockCount = 10;
}

// Lớp xâm phạm (Content Coupling)
class OrderService {
    public void processOrder(Inventory inv, int amount) {
        // Thay đổi trực tiếp trạng thái nội bộ của Inventory
        // Thay vì gọi phương thức inv.reduceStock(amount)
        if (inv.stockCount ≥ amount) {
            inv.stockCount -= amount;
            System.out.println("Đặt hàng thành công!");
        }
    }
}
```


Remove Content Coupling

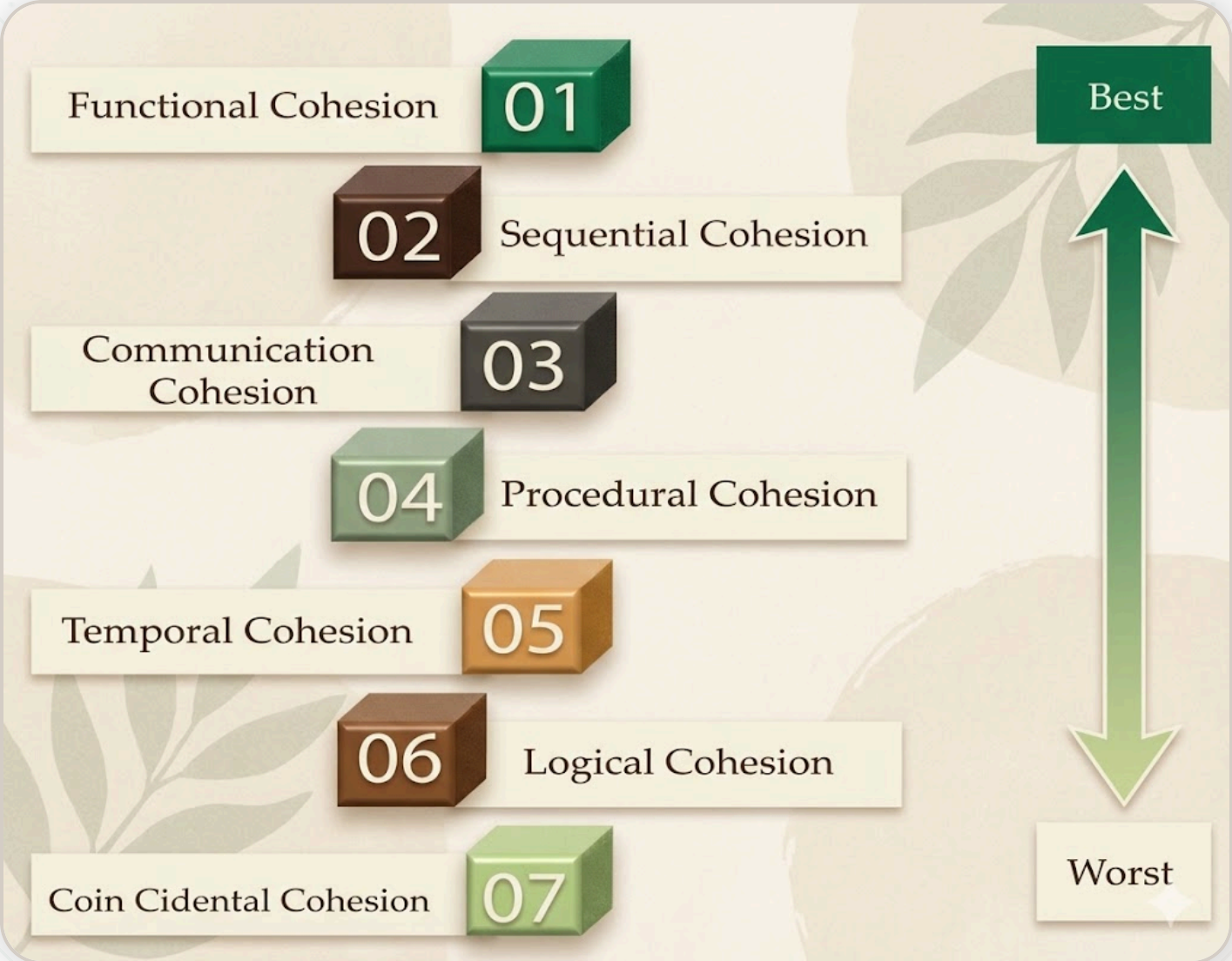
```
class Inventory {  
    private int stockCount = 10; // Chuyển sang private  
    public boolean reduceStock(int amount) {  
        if (amount ≤ stockCount) {  
            stockCount -= amount;  
            return true;  
        }  
        return false;  
    }  
}  
  
class OrderService {  
    public void processOrder(Inventory inv, int amount) {  
        if (inv.reduceStock(amount)) {  
            System.out.println("**Đặt hàng thành công!**);  
        }  
    }  
}
```

Module Cohesion

- ☕ Là thước đo mức độ **liên quan** giữa các thành phần bên trong một mô-đun.
- ☕ **Mục tiêu:** Kết dính cao (*High cohesion*).
 - 🌿 Một mô-đun chỉ nên làm một việc, và làm thật tốt việc đó.
- ☕ **Lợi ích:**
 - 🌿 Mô-đun dễ hiểu và có chức năng tập trung.
 - 🌿 Dễ bảo trì và kiểm thử.
 - 🌿 Tăng khả năng tái sử dụng vì chức năng đã được đóng gói gọn gàng.

Các loại Kết dính (Cohesion)

- 1 **Functional:** Hoàn hảo, mọi phần tử bên trong phục vụ một mục tiêu duy nhất.
- 2 **Sequential:** Dây chuyền: Kết quả bước 1 là đầu vào bước 2.
- 3 **Communicational:** Các hàm dùng chung tập dữ liệu (VD: Các hàm trong `UserDAO`).
- 4 **Procedural:** Các bước thực hiện theo trình tự thời gian nhưng không chung dữ liệu.
- 5 **Temporal:** Gom các hàm chạy cùng lúc (VD: `Initialize()` , `InitDatabase()` , `InitUI()`).
- 6 **Logical:** Gom các hàm có vẻ giống loại (VD: Tất cả hàm in ấn, tất cả hàm xử lý lỗi).
- 7 **Coincidental:** Gom ngẫu nhiên, không có logic liên kết (đây là thiết kế **tệ nhất**).



COUPLING vs COHESION

Tiêu chí	Coupling (Kết ghép)	Cohesion (Kết dính)
Phạm vi	Giữa các mô-đun khác nhau	Bên trong một mô-đun
Mục tiêu	Càng thấp càng tốt (Minimize)	Càng cao càng tốt (Maximize)
Nội dung	Đo lường sự phụ thuộc	Đo lường mục đích chức năng
Ưu tiên	Loose Coupling	High Cohesion

"Loose Coupling, High Cohesion" là kim chỉ nam cho mọi kiến trúc phần mềm tốt.

FUNCTION ORIENTED DESIGN (FOD)

Tư duy: Hệ thống được nhìn nhận như một bộ máy khổng lồ, nơi dữ liệu đi vào ở một đầu, trải qua các hàm biến đổi và đi ra ở đầu kia dưới dạng thông tin có ích.



Đặc điểm:

- ✱ Lấy hành động làm trung tâm: Tập trung trả lời câu hỏi Hệ thống này làm gì? (**What does it do?**) thay vì Hệ thống này quản lý đối tượng nào?
- ✱ Phân rã chức năng (**Top-down**): Bắt đầu từ một chức năng tổng quát nhất, sau đó chia nhỏ dần thành các hàm con đơn giản hơn cho đến khi có thể lập trình được.

FUNCTION ORIENTED DESIGN (FOD)

- ☕ **Tối ưu:** Các hệ thống **xử lý giao dịch tập trung** (như tính lương, xử lý hóa đơn) hoặc các phần mềm tính toán kỹ thuật phức tạp.
- ☕ **Hạn chế:** Khi yêu cầu về dữ liệu thay đổi, ta thường phải sửa lại rất nhiều hàm liên quan, dẫn đến chi phí bảo trì cao trong các dự án lớn và dài hạn.

Design Notations - Data Flow Diagram (DFD)

- ☕ **Mục tiêu:** Chỉ ra cách dữ liệu di chuyển và bị biến đổi trong hệ thống.
- ☕ **Thành phần:** Process (Tiến trình), Data Store (Kho dữ liệu), Source/Sink (Nguồn/Đích), Data Flow (Luồng).
- ☕ **Ưu điểm:** Trực quan, dễ thảo luận với khách hàng không am hiểu kỹ thuật.



Design Notations - Data Dictionaries

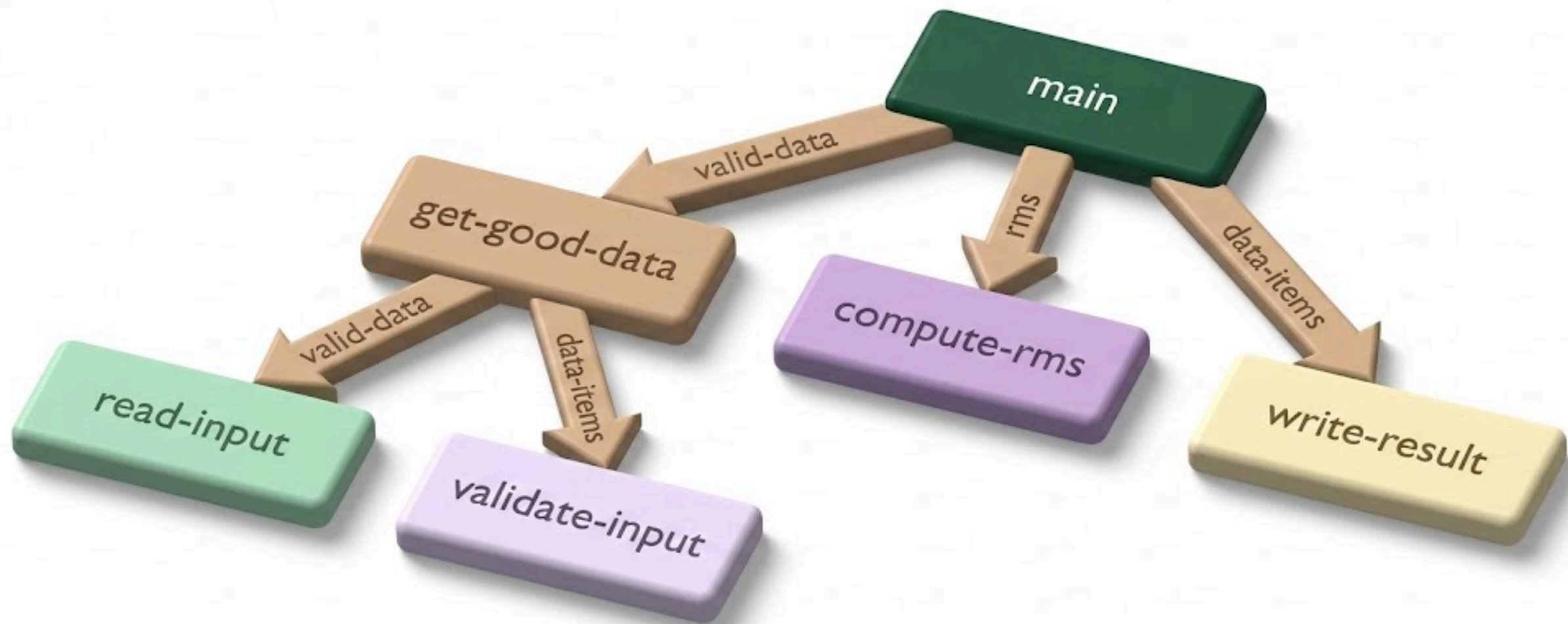
-  **Vai trò:** Nếu DFD là bản đồ thì Từ điển dữ liệu chính là cuốn **bách khoa toàn thư** giải thích mọi thuật ngữ trên bản đồ đó
-  **Nội dung:** Định nghĩa tên, kiểu dữ liệu, phạm vi giá trị và cấu trúc.
 -  Biến: `Customer_Phone`
 -  Định nghĩa: Số điện thoại liên lạc của khách hàng tại Việt Nam.
 -  Định dạng: Chuỗi số, độ dài 10 ký tự, bắt đầu bằng số 0.
-  **Lợi ích:** Loại bỏ tình trạng một biến nhưng mỗi lập trình viên hiểu theo một kiểu khác nhau, giúp việc tích hợp các mô-đun diễn ra trơn tru.

Design Notations - Structure Charts

- 🍷 **Bản đồ phân cấp của hệ thống:** Nếu DFD cho thấy dữ liệu đi đâu, thì **Structure Chart** cho thấy *ai là người điều hành*. Nó mô tả mối quan hệ cha-con giữa các mô-đun.
- 🍷 Các ký hiệu đặc trưng:
 - 🌿 **Hình chữ nhật:** Đại diện cho một mô-đun (hàm, thủ tục hoặc lớp).
 - 🌿 **Mũi tên kết nối:** Chỉ ra mô-đun cha đang gọi (invoke) mô-đun con.
 - 🌿 **Mũi tên nhỏ kèm vòng tròn rỗng:** Đại diện cho Data Couples (Dữ liệu được truyền đi).
 - 🌿 **Mũi tên nhỏ kèm vòng tròn đặc:** Đại diện cho Control Couples (Cờ điều khiển/Flag).
- 🍷 **Giá trị cốt lõi:** Giúp người thiết kế nhìn thấy ngay lập tức nếu một mô-đun đang phải nhận quá nhiều cờ điều khiển (**Control Coupling**), từ đó có kế hoạch tái cấu trúc kịp thời.

Design Notations - Structure Charts

- ☕ **Đánh giá độ phức tạp:** Nhìn vào biểu đồ, ta có thể thấy được Fan-in (có bao nhiêu mô-đun gọi nó) và Fan-out (nó gọi bao nhiêu mô-đun khác).
- ☕ **Tăng tính độc lập:** Giúp phát hiện các mô-đun đang ôm đồm quá nhiều việc, thúc đẩy việc chia nhỏ để đạt được High Cohesion.
- ☕ **Khác biệt với Lưu đồ (Flowchart):** Structure Chart tập trung vào cấu trúc tĩnh và sự phân cấp, không sa đà vào chi tiết các bước thực hiện bên trong như **Flowchart**.



Design Notations - Pseudo-Code

- ☕ Là sự kết hợp giữa ngôn ngữ tự nhiên và cấu trúc logic của lập trình. Nó không có cú pháp khắt khe như **Java** hay **Python** nhưng vẫn rất chặt chẽ về mặt tư duy.
- ☕ Tại sao nên dùng mã giả trước khi viết code?
 - 🌿 **Tập trung vào giải thuật:** Giúp lập trình viên giải quyết bài toán logic hóc búa mà không bị phân tâm bởi việc thiếu dấu chấm phẩy hay sai kiểu dữ liệu.
 - 🌿 **Dễ dàng trao đổi:** Một BA có thể đọc hiểu mã giả để xác nhận logic với khách hàng mà không cần biết lập trình.
 - 🌿 **Khả năng chuyển đổi:** Từ một bản mã giả tốt, việc chuyển sang bất kỳ ngôn ngữ lập trình nào cũng trở nên cực kỳ nhanh chóng.

Design Notations - Pseudo-Code

Bài toán: Kiểm tra và xử lý thanh toán.

```
BEGIN
    Nhập số tiền và số dư tài khoản
    IF số tiền > số dư THEN
        Thông báo: Không đủ số dư
        Trạng thái = THẤT BẠI
    ELSE
        Trừ tiền trong tài khoản
        Gửi email xác nhận
        Trạng thái = THÀNH CÔNG
    END IF
    RETURN Trạng thái
END
```


OBJECT-ORIENTED DESIGN

Tư duy: Hệ thống là một cộng đồng các **đối tượng** cùng hợp tác.

- ☕ **Đối tượng (Object):** Thành phần kết hợp cả dữ liệu (state) và hành vi (behavior).
- ☕ **Lợi ích cốt lõi:**
 - 🌿 Khả năng **tái sử dụng** cao nhờ kế thừa và mô-đun hóa.
 - 🌿 Dễ dàng ánh xạ từ thế giới thực vào phần mềm.
 - 🌿 **Dễ bảo trì:** Thay đổi bên trong một lớp ít ảnh hưởng đến các lớp khác (**Encapsulation**).

Các khái niệm then chốt trong OOD

- ☕ **Abstraction:** Tập trung vào các thuộc tính quan trọng, bỏ qua chi tiết cài đặt.
- ☕ **Encapsulation (Đóng gói):** Giấu dữ liệu bên trong, chỉ cho phép truy cập qua *giao diện* (phương thức).
- ☕ **Inheritance (Kế thừa):** Cho phép lớp con sở hữu các đặc điểm của lớp cha, giúp tránh lặp mã (**DRY** - Don't Repeat Yourself).
- ☕ **Polymorphism (Đa hình):** Một `message` có thể được xử lý theo nhiều cách khác nhau tùy vào đối tượng nhận.
 - 🌿 VD: Một hàm `draw()` nhưng hình tròn vẽ khác hình vuông.

USER INTERFACE DESIGN



Giao diện dòng lệnh (CLI - Command Line Interface):



Đối tượng: Chuyên gia, kỹ sư hệ thống.



Ưu: Tốc độ thực thi cực nhanh, tiết kiệm tài nguyên, dễ tự động hóa (scripting).



Nhược: Khó học, người dùng phải nhớ chính xác câu lệnh và cú pháp.



Giao diện đồ họa (GUI - Graphical User Interface):



Đối tượng: Mọi tầng lớp người dùng.



Ưu: Trực quan, dễ khám phá, tận dụng các phép ẩn dụ thực tế (Icon thùng rác, Folder).



Nhược: Tốn bộ nhớ/GPU, khó tự động hóa bằng câu lệnh.

Các nguyên lý thiết kế UI

- 🍷 **Cấu trúc (Structure):** Tổ chức giao diện có ý nghĩa (VD: thanh menu ở trên, nội dung ở giữa).
- 🍷 **Đơn giản (Simplicity):** Chỉ hiển thị những gì cần thiết. Giảm tải nhận thức (cognitive load).
- 🍷 **Khả năng nhìn thấy (Visibility):** Các chức năng quan trọng không được ẩn quá sâu (Quy tắc 3 click).
- 🍷 **Phản hồi (Feedback):** Hệ thống luôn phải báo cho người dùng biết chuyện gì đang xảy ra (VD: Thanh tiến trình, âm báo).
- 🍷 **Sự khoan dung (Tolerance):** Cho phép người dùng sai lầm (VD: Nút **Undo**, cảnh báo **Bạn có chắc chắn muốn xóa?**).

GIAI ĐOẠN VIẾT MÃ (CODING)

- ☕ **Mô tả:** Chuyển hóa bản vẽ kỹ thuật (Design) thành mã nguồn thực thi.
- ☕ **Mục tiêu ưu tiên:**
 - 🌿 **Tính chính xác:** Code phải chạy đúng yêu cầu (Logic).
 - 🌿 **Tính dễ đọc (Readability):** Viết code cho con người đọc, máy tính chỉ là phụ?
 - 🌿 **Tính bảo trì:** Code dễ sửa, dễ thêm tính năng mà không làm hỏng cái cũ.
 - 🌿 **Hiệu suất:** Tối ưu tốc độ và bộ nhớ (nhưng không được đánh đổi sự rõ ràng một cách mù quáng).

ĐẶC TÍNH CỦA NGÔN NGỮ LẬP TRÌNH TỐT

- ☕ **Readability:** Cú pháp rõ ràng, gần với ngôn ngữ tự nhiên (VD: Python).
- ☕ **Portability:** Code chạy được trên nhiều nền tảng (VD: Java - Write once, run everywhere).
- ☕ **Generality:** Có khả năng giải quyết nhiều loại vấn đề.
- ☕ **Error Checking:** Hỗ trợ phát hiện lỗi sớm (Strong typing, static analysis).
- ☕ **Efficiency:** Khả năng tối ưu hóa của trình biên dịch.
- ☕ **Modularity:** Hỗ trợ tốt việc chia nhỏ mã thành các thư viện/class.

CODING STANDARDS

Mã nguồn là tài sản chung của nhóm, không phải của riêng bạn.



Vì sao cần chuẩn mực?

- 🌿 Giảm công sức khi đọc code của nhau.
- 🌿 Tránh các lỗi ngớ ngẩn (VD: quên xử lý `null`).

CODING STANDARDS

- ☕ **Các thành phần chính:**
 - 🌿 **Đặt tên (Naming):** `camelCase`, `PascalCase`, `snake_case`. Tên biến phải nói lên mục đích.
 - 🌿 **Thụt đầu dòng:** Thống nhất dùng Tab hoặc Space.
 - 🌿 **Bình luận (Comments):** Tập trung giải thích **TẠI SAO** (logic nghiệp vụ), đừng giải thích **CÁI GÌ** (cú pháp ngôn ngữ).
 - 🌿 **Cấu trúc:** Giới hạn độ dài một file và một hàm.

Nguyên lý viết code sạch (Guidelines)

- ☕ **Độ dài dòng:** Thường < 80-120 ký tự (để không phải cuộn ngang).
- ☕ **Khoảng trắng (White space):** Dùng để phân tách các khối logic, giúp mắt người dễ quét.
- ☕ **Hàm (Functions):** Nên làm một việc duy nhất (Single Responsibility).
- ☕ **Side-effects:** Hạn chế việc một hàm thay đổi dữ liệu bên ngoài mà không báo trước.
- ☕ **Thông báo lỗi:** Phải mang tính xây dựng, giúp người dùng/LTV biết cách khắc phục.

PROGRAMMING STYLE

- ☕ **Tư tưởng:** Code không chỉ để máy tính hiểu, mà còn là phương tiện giao tiếp giữa các lập trình viên.
- ☕ **Các quy tắc cốt yếu:**
 - 🌿 **Tên biến/hàm:** Mang tính mô tả (VD: `customerBalance` thay vì `b`).
 - 🌿 **Cấp độ lồng nhau (Nesting):** Tránh các cấu trúc "mũi tên" (lồng lặp/điều kiện quá sâu). Sử dụng *Guard Clauses* để thoát hàm sớm.
 - 🌿 **Single Entry - Single Exit:** Một hàm nên có một điểm vào và điểm ra rõ ràng (dù `return` sớm đôi khi vẫn được chấp nhận nếu giúp code sạch hơn).
 - 🌿 **Che giấu thông tin:** Không lộ các biến nội bộ (private) ra bên ngoài nếu không thực sự cần thiết.

STRUCTURED PROGRAMMING

- ☕ **Mục tiêu:** Tạo ra luồng điều khiển tuyến tính, dễ dự đoán.
- ☕ **5 Cấu trúc chuẩn:**
 - 🌿 **Code Block:** Các câu lệnh thực thi tuần tự.
 - 🌿 **Sequence:** Kết nối các khối mã.
 - 🌿 **Alternation:** Rẽ nhánh logic (`If-Else` , `Switch-Case`).
 - 🌿 **Iteration:** Vòng lặp kiểm soát được (`For` , `While` , `Do-While`).
 - 🌿 **Nested Structures:** Kết hợp các cấu trúc trên một cách khoa học.
- ☕ **Nghiêm cấm:** Sử dụng lệnh `GOTO` gây rối loạn luồng điều khiển (Spaghetti code).

SOFTWARE RELIABILITY

-  **Định nghĩa:** Xác suất hệ thống hoạt động không lỗi trong một khoảng thời gian nhất định và môi trường nhất định.
-  **Thực trạng:** Khi phần mềm ngày càng lớn (hàng triệu dòng code), xác suất lỗi tiềm tàng tăng theo cấp số nhân.
-  **Giải pháp tăng độ tin cậy:**
 -  Áp dụng các nguyên lý thiết kế chặt chẽ (Coupling/Cohesion).
 -  Kiểm thử (Testing) toàn diện từ Unit đến System Test.
 -  Sử dụng các công cụ phân tích mã tĩnh (Static Analysis).
 -  Ví dụ điển hình: Hệ thống điều khiển Boeing, phần mềm Trạm vũ trụ.

SOFTWARE DESIGN in AI ERA

Thiết kế phần mềm & Agentic Coding

- ☕ AI Agent không chỉ gợi ý code mà đã có thể tự thực hiện các Task phức tạp.
 - 🌱 AI có thể tạo ra hàng nghìn dòng code trong vài phút.
- ☕ **Nguy cơ:** Nếu không có thiết kế tốt, AI sẽ tạo ra **nợ kỹ thuật (Technical Debt) tốc độ cao** – một đống code chạy được nhưng không thể bảo trì hoặc mở rộng.
- ☕ Thiết kế đóng vai trò là bản chỉ dẫn (Blueprint) để điều hướng AI, đảm bảo kết quả đầu ra tuân thủ đúng cấu trúc hệ thống.

Thiết kế phần mềm & Agentic Coding

1 Quản lý sự phức tạp (**Complexity Management**):

- AI giỏi giải quyết các hàm cục bộ, nhưng con người cần thiết kế để giữ cho các mô-đun không bị chồng chéo.
- Duy trì Loose Coupling** để khi AI thay đổi một mô-đun, nó không làm hỏng toàn bộ hệ thống.

2 Interface as a Contract

- Thiết kế chi tiết các Interface giúp AI biết chính xác nó cần **lắp ghép** mã nguồn vào đâu.
- Giúp việc kiểm thử tự động (**Unit Test**) cho code do AI tạo ra trở nên khả thi.

3 Khả năng kiểm chứng (Verifiability):

- Code sạch, có cấu trúc giúp **con người** (hoặc một **Agent khác**) dễ dàng review

Vai trò của lập trình viên

- ☕ Từ **Người thợ xây** sang **Kiến trúc sư**:
 - 🌿 Thay vì tập trung vào cú pháp (**Syntax**), LTV tập trung vào trừu tượng hóa (**Abstraction**) và phân chia vấn đề (**Partitioning**).
- ☕ Thiết kế là ngôn ngữ giao tiếp với AI:
 - 🌿 Một tài liệu thiết kế (SDD) tốt chính là một **Prompt** khổng lồ và chính xác nhất cho AI Agent.
 - 🌿 AI thực thi câu hỏi **What to write**, nhưng con người phải quyết định **How to structure**.

Nguyên tắc thiết kế cho kỹ nguyên AI

- 🍷 **Modularity Pro Max:** chia nhỏ hệ thống thành các phần cực kỳ độc lập để AI có thể làm việc trên từng phần mà không cần hiểu toàn bộ triệu dòng code.
- 🍷 **Spec-Driven Development:** tài liệu thiết kế, sơ đồ DFD, Class Diagram trở thành **Source of Truth** để AI hiểu ngữ cảnh dự án.
- 🍷 **Design for Testability:** thiết kế hệ thống sao cho mọi thành phần đều dễ dàng được kiểm soát bởi các kịch bản test tự động.

TỔNG KẾT BÀI HỌC

- ☕ **Cốt lõi:** chuyển hóa từ "Cái gì" (What) sang "Thế nào" (How) dựa trên 3 trụ cột: **Phân chia, Trừu tượng hóa & Mô-đun hóa.**
- ☕ **Chỉ số chất lượng:** luôn ưu tiên **Kết ghép lỏng (Loose Coupling)** và **Kết dính cao (High Cohesion)** để đảm bảo tính linh hoạt và dễ kiểm soát.
- ☕ **Kỷ nguyên Agentic Coding:**
 - 🌿 Thiết kế là **điểm neo** duy nhất giúp kiểm soát chất lượng khi AI tự động sinh mã hàng loạt.
 - 🌿 Tài liệu thiết kế (SDD) là **bản chỉ dẫn** chuẩn mực nhất để điều hướng các **AI Agent**.
- ☕ **Tính bền vững:** mã nguồn sạch và chuẩn mực là điều kiện tiên quyết để hệ thống có thể bảo trì lâu dài bởi cả con người và AI.

Q&A

CẢM ƠN ĐÃ THEO DÕI